

Guia tecnico do codigo

Zeladoria Coopatos

Resumo didatico para apresentar ao professor: arquitetura, dependencias, arquivos, fluxos e perguntas comuns.

Objetivo do sistema

Registrar, acompanhar e resolver chamados de zeladoria com midias, localizacao, chat, notificacoes e painel administrativo.

Publico

Funcionarios abrem e acompanham chamados. Administradores gerenciam equipe, chamados, indicadores, mapa e auditoria.

Stack central

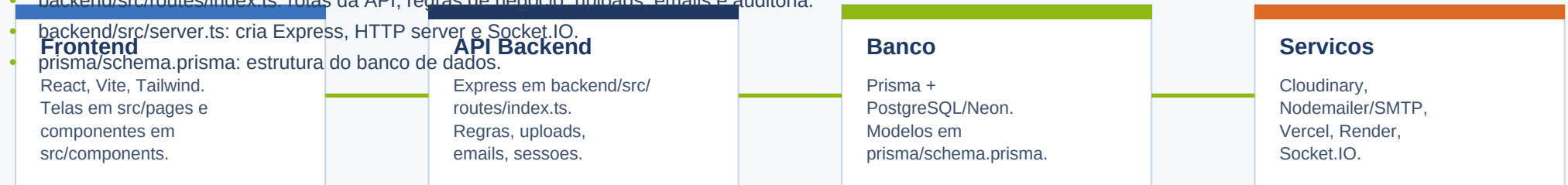
React + Vite no frontend, Express no backend, Prisma/PostgreSQL no banco e Socket.IO para tempo real.

1. Arquitetura geral

A aplicação é dividida em frontend, backend, banco de dados e serviços externos. O frontend mostra as telas e chama a API. O backend valida regras, grava no banco e dispara eventos em tempo real. O banco guarda usuários, chamados, mensagens, notificações, auditoria e mídias.

Arquivos para citar

- `src/App.tsx`: define as rotas principais e proteções de funcionário/admin.
- `src/pages/EmployeePanel.tsx`: painel do funcionário, chamados, chats e notificações.
- `src/pages/Dashboard.tsx`: painel administrativo, indicadores, mapa, CRUDs e exportações.
- `backend/src/routes/index.ts`: rotas da API, regras de negócio, uploads, emails e auditoria.
- `backend/src/server.ts`: cria Express, HTTP server e Socket.IO.
- `prisma/schema.prisma`: estrutura do banco de dados.



2. Dependencias principais

Na apresentacao, foque nas tecnologias importantes. Existem varias libs de UI, mas estas sao as que explicam a arquitetura e as funcionalidades principais.

React + TypeScript + Vite

Interface web, componentes e build do frontend. Local: src/main.tsx, src/App.tsx e src/pages.

Tailwind + shadcn/Radix

Estilo visual, modais, inputs, selects, botoes e toasts. Local: src/components/ui e src/index.css.

Node + Express

API HTTP do sistema. Local: backend/src/server.ts e backend/src/routes/index.ts.

Prisma + PostgreSQL/Neon

ORM e banco relacional. Local: prisma/schema.prisma e backend/src/prisma/client.ts.

Socket.IO

Tempo real: chats, notificacoes, presenca e logout forçado. Local: server.ts, EmployeePanel.tsx e Dashboard.tsx.

Cloudinary

Upload/armazenamento de fotos, videos e audios. Local: EmployeePanel.tsx, Dashboard.tsx e routes/index.ts.

Nodemailer

Emails de validacao e recuperacao. Local: backend/src/routes/index.ts.

Leaflet + OpenStreetMap

Mapa com pins e mapa de calor. Local: src/components/DashboardMaps.tsx.

Recharts

Graficos e indicadores administrativos. Local: src/pages/Dashboard.tsx.

3. Frontend: telas e rotas

O frontend e uma SPA. O React Router decide qual tela abrir. As telas protegidas usam AuthContext para conferir se o usuario esta autenticado e se e funcionario ou administrador.

src/App.tsx

```
Route path="/" -> EmployeeLogin
Route path="/admin/login" -> AdminLogin
Route path="/funcionario" -> ProtectedEmployee + EmployeePanel
Route path="/dashboard" -> ProtectedAdmin + Dashboard
Route path="/validar-email/:token" -> VerifyEmployeeEmail
```

Como explicar

- EmployeeLogin.tsx faz login por matricula e CPF e salva os dados da sessao.
- AdminLogin.tsx faz login administrativo por email e senha.
- EmployeePanel.tsx concentra o fluxo do funcionario: criar chamado, listar reportes, chat, privado, notificacoes e midias.
- Dashboard.tsx concentra o admin: indicadores, chamados, mapa, funcionarios, usuarios, departamentos, auditoria e whitelabel.
- AuthContext.tsx centraliza login, logout, inatividade e persistencia no navegador.

4. Backend/API: onde ficam as regras

O backend é um servidor Express. Quase todas as rotas estão em `backend/src/routes/index.ts`. Ele recebe requisições do frontend, usa Prisma para consultar/gravar no banco, integra serviços externos e emite eventos Socket.IO quando precisa atualizar a tela em tempo real.

`backend/src/server.ts`

```
const app = express()
const server = http.createServer(app)
export const io = new Server(server, { cors: ... })
socket.join(`employee-${employeeId}`)
socket.on("join-report", reportId => socket.join(`report-${reportId}`))
```

Rotas importantes para citar

- POST `/employee-login`: login do funcionário.
- POST `/admin-login`: login administrativo.
- GET/POST/PATCH/DELETE `/reports`: listagem, criação, edição e exclusão de chamados.
- `/reports/:id/messages`: chat do chamado.
- `/private-conversations`: chat privado.
- `/notifications/:employeeId`: notificações.
- `/admin/employees`, `/admin/departments`, `/admin/users`: CRUDs administrativos.
- `/admin/audit-logs` e `/admin/exports/:type`: auditoria e exportações.

5. Banco de dados: modelos principais

O Prisma define as tabelas e relacionamentos. Se o professor perguntar onde estão as tabelas, a resposta é prisma/schema.prisma. As migrations ficam em prisma/migrations.

Employee

Funcionario: matricula, CPF, email, departamento, foto, sessao ativa.

User

Usuario administrativo e permissao de acesso.

Report

Chamado principal: categoria, status, prioridade, localizacao e dono.

ReportImage

Mídias anexadas ao chamado.

ReportMessage

Mensagens do chat do chamado.

PrivateConversation

Conversa privada entre funcionarios.

Notification

Notificacoes de mensagens, mencoes e atribuicoes.

AuditLog

Registro de alteracoes administrativas.

ReportNote

Anotacoes/observacoes internas no chamado.

6. Chats: implementacao e fluxo

Os chats usam a mesma ideia: o frontend envia a mensagem para a API, o backend salva no banco com Prisma e depois emite um evento Socket.IO para os usuarios conectados receberem a mensagem em tempo real.

Onde esta cada parte

- Frontend dos chats: src/pages/EmployeePanel.tsx.
- Socket.IO backend: backend/src/server.ts.
- Rotas de mensagens: backend/src/routes/index.ts.
- Modelos do banco: ReportMessage, PrivateMessage, ReportMessageRead, PrivateMessageRead, Notification.
- Anexos: upload para Cloudinary antes de salvar a mensagem com URL da midia.
- Check para mark-read grava ate qual mensagem foi lida e o Socket.IO avisa os outros usuarios
- emoji, resposta e /reports/:id/messages
- Ptes da mensagem, midias, mencoes e

4. Socket.IO

Backend emite new-message ou private-

Eventos para mencionar

join-report, leave-report, new-message, messages-read
join-private-conversation, private-message, private-messages-read
notification-created, force-logout

7. Painel do funcionario

O painel do funcionario esta praticamente todo em EmployeePanel.tsx. Ele tem varias funcoes porque concentra criacao de chamado, reportes, chat do chamado, chat privado, notificacoes, filtros, localizacao e upload.

- Criar chamado: formulario com titulo, categoria, prioridade, descricao, endereco/GPS e midia.
- Meus Reportes: lista chamados do usuario, ordena pendentes antes de finalizados e por prioridade.
- Reportes: mostra chamados em que o funcionario participa.
- Chat do chamado: conversa vinculada ao chamado, com midias, emoji, resposta e leitura.
- Chat privado: conversa um a um, com selecao, exclusao, resposta, midias e check azul.
- Notificacoes: painel mostra mencoes, mensagens privadas e atualizacoes.
- Inatividade e logout: AuthContext controla timeout e Socket.IO trata nova sessao em outro local.

Arquivos para citar

src/pages/EmployeePanel.tsx

src/contexts/AuthContext.tsx

backend/src/routes/index.ts

prisma/schema.prisma

8. Dashboard administrativo

O Dashboard.tsx e a tela administrativa. Ele consome as rotas admin do backend, mostra dados filtrados e permite administrar o sistema. A area administrativa tambem e protegida: apenas usuario ligado ao departamento administrativo entra.

- Indicadores: cards, graficos e filtros proprios. Usa Recharts.
- Chamados: lista, filtra, cria, edita, exclui, atribui funcionarios e exporta CSV/PDF.
- Mapa: componente DashboardMaps.tsx com Leaflet, pins e mapa de calor.
- Funcionarios: CRUD em modal, foto, nascimento, departamento, validacao de email e status ativo/inativo.
- Usuarios e Departamentos: filtros, alteracao de departamento, ativar/desativar e auditoria.
- Auditoria: consulta AuditLog, mostra quem fez a alteracao e exporta relatorios.
- Personalizacao: muda nome, logo e cores usando src/config/brand.ts.

Arquivos para citar

src/pages/Dashboard.tsx

src/components/DashboardMaps.tsx

src/config/brand.ts

backend/src/routes/index.ts

9. Integracoes externas

As integracoes sao importantes porque mostram que o sistema nao e apenas tela: ele envia arquivos, emails, usa mapa e tempo real.

- Cloudinary: O frontend envia o arquivo e recebe uma URL. A API salva essa URL no banco. Usado em chamados, chat, anotacoes e foto de perfil.
- Nodemailer/SMTP: Backend envia email de validacao de cadastro e recuperacao de matricula. Templates usam backend/src/config/brand.ts.
- Socket.IO: Mantem tempo real: chats, notificacoes, presenca online, leitura e logout forçado.
- Leaflet/OpenStreetMap: Mostra chamados no mapa. DashboardMaps.tsx usa coordenadas dos chamados filtrados.
- Vercel/Render/Neon: Frontend fica na Vercel, API no Render e banco PostgreSQL no Neon.
- Whitelabel: Configura nome, logo e cores. Frontend aplica variaveis CSS e backend usa o preset nos emails.

10. Perguntas provaveis e respostas curtas

P: Onde ficam as rotas?

R: backend/src/routes/index.ts. O server.ts apenas cria Express e Socket.IO.

P: Onde ficam as tabelas?

R: prisma/schema.prisma. As alteracoes do banco ficam em prisma/migrations.

P: Como o chat e em tempo real?

R: A mensagem e salva pela API e o backend emite evento Socket.IO para a sala do chamado ou conversa privada.

P: Como salva imagem/video/audio?

R: O arquivo vai para Cloudinary, volta uma URL e essa URL e salva no banco.

P: Como funciona o check azul?

R: Ao abrir a conversa, o frontend chama mark-read; o backend grava a ultima mensagem lida e emite evento de leitura.

P: Como o admin ve indicadores?

R: Dashboard.tsx filtra os chamados e monta dados para Recharts.

P: Como o mapa funciona?

R: DashboardMaps.tsx usa Leaflet e coordenadas dos chamados para pins e heatmap.

P: Como protege o admin?

R: App.tsx exige role admin e departamento administrativo; backend valida sessao nas rotas /admin.

P: Como foi feito whitelabel?

R: src/config/brand.ts define preset e aplica variaveis CSS; backend/src/config/brand.ts personaliza emails.

11. Cola final de apresentacao

Use este discurso curto se precisar explicar o projeto inteiro em menos de um minuto:

Discurso sugerido

O projeto e uma SPA em React/TypeScript com backend Node/Express.
O frontend fica em src/pages e o backend em backend/src/routes/index.ts.
Os dados sao salvos em PostgreSQL/Neon usando Prisma, definido em prisma/schema.prisma.
Chamados, chats, notificacoes e logout usam Socket.IO para tempo real.
Uploads de midia usam Cloudinary, emails usam Nodemailer e mapas usam Leaflet.
O painel admin em Dashboard.tsx gerencia chamados, funcionarios, indicadores, mapa e auditoria.
O painel do funcionario em EmployeePanel.tsx cria chamados e concentra os chats.

Dependencias para lembrar

- React, TypeScript, Vite
- Tailwind, shadcn/ui, Radix UI
- Node.js, Express
- Prisma, PostgreSQL/Neon
- Socket.IO
- Cloudinary
- Nodemailer
- Leaflet
- Recharts
- Vercel, Render